

COMP521 Assignment 3 - Mean Elapsed Time by Worker Count

Prediction

For a large CPU-bound workload, increasing the number of workers should reduce elapsed time. Although the elapsed time will be reduced, the speedup alone should not be considered an improvement as there are other factors outside of the program's control, such as thread creation, scheduling, synchronization, swapping, and other system overhead.

Environment

This experiment was conducted on a Framework Laptop 13 running Arch Linux. The OS and hardware information will be provided by the output of `uname -a`, `nproc`, and `lscpu`

```
uname -a
```

```
Linux Briskycola-FWL 7.2.7-arch1-1 #1 SMP PREEMPT_DYNAMIC Mon, 21 Sep 2026 18:51:14 +0000 x86_64
```

```
nproc
```

```
12
```

```
lscpu | head -n 20
```

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:         48 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                12
On-line CPU(s) list:   0-11
Vendor ID:             AuthenticAMD
Model name:            AMD Ryzen 5 7640U w/ Radeon 760M Graphics
CPU family:            25
Model:                 116
Thread(s) per core:    2
Core(s) per socket:    6
Socket(s):             1
Stepping:              1
Microcode version:     0xa70410a
Frequency boost:       enabled
CPU(s) scaling MHz:    28%
CPU max MHz:           4972.4351
CPU min MHz:           405.9140
BogoMIPS:              6986.91
```

Experimental Design

In this experiment, the independent variable was the number of workers used to complete the computation. The tested worker counts were 1, 2, and 4. The dependent variable was elapsed time in seconds. The elapsed time was measured using `clock_gettime()` with `CLOCK_MONOTONIC` in C. The total workload was controlled at 200,000,000 work units for every condition. The workload was divided among the workers, but the total amount of work remained constant. Three trials were completed for each worker count, resulting in nine total measurements. The worker function performed a CPU-bound calculation for each assigned work unit. The calculation performs a bitwise XOR operation of two extremely large values. The left hand side is an integer multiplied by 31 and the right hand side is the same integer shifted right by three bits.

Result

The experiment produced three trials for each worker count. The total workload was 200,000,000 work units in every trial.

Raw Trial Timing Data

Workers	Trial	Work Units	Elapsed (s)
1	1	200000000	0.135938
1	2	200000000	0.135094
1	3	200000000	0.134304
2	1	200000000	0.067799
2	2	200000000	0.067760
2	3	200000000	0.069482
4	1	200000000	0.034791
4	2	200000000	0.034364
4	3	200000000	0.034471

Timing Summary

Workers	Mean Time (s)	Minimum (s)	Maximum (s)	Range (s)
1	0.135112	0.134304	0.135938	0.001634
2	0.068347	0.067760	0.069482	0.001722
4	0.034542	0.034364	0.034791	0.000427

Speedup and Efficiency

The one-worker mean was used as the baseline.

Workers	Mean Time (s)	Speedup	Efficiency
1	0.135112	1.000000	100.00%
2	0.068347	1.976853	98.84%
4	0.034542	3.911528	97.79%

The graph titled “Mean Elapsed Time by Worker Count” (lab5a_graph.pdf) shows that mean elapsed time decreased as the number of workers increased.

Evidence-Based Analysis

Evidence

The one-worker condition had a mean elapsed time of 0.135112 seconds. The two-worker condition had a mean elapsed time of 0.068347 seconds, and the four-worker condition had a mean elapsed time of 0.034542 seconds.

Compared with one worker, two workers produced a speedup of approximately 1.98 and an efficiency of approximately 98.84%. Four workers produced a speedup of approximately 3.91 and an efficiency of approximately 97.79%.

The trial ranges were small. The one-worker range was 0.001634 seconds, the two-worker range was 0.001722 seconds, and the four-worker range was 0.000427 seconds. This would indicate that there is in fact a performance boost with more workers. However, the results do not indicate the speedup to be at **exactly** the ideal value. Although there is a visible performance improvement when dividing the workload among multiple workers, the speedup was not quite at the ideal value due to serial work still being a factor along with other factors such as scheduling, synchronization, and general system overhead.

Meaning

Under the recorded conditions, increasing the number of workers reduced the time required to complete the fixed workload. The four-worker condition was the fastest tested condition, completing the workload in approximately one-fourth of the one-worker time.

The results were very close, but not quite at ideal scaling. Doubling the workers from one to two produced almost a 2x speedup. Increasing the workers from one to four produced a speedup of approximately 3.91x, which is close to the ideal 4x speedup.

The small trial ranges indicate that the measurements were relatively consistent during the experiment, although the two-worker condition had slightly more variation than the other conditions.

Connection

The result demonstrates strong scaling because the total workload remained fixed while the number of workers changed. Dividing the work among multiple workers allowed portions of the CPU-bound computation to execute concurrently.

The speedup was slightly below ideal: 1.98 instead of 2.00 for two workers and 3.91 instead of 4.00 for four workers. This difference is consistent with thread creation, scheduling, synchronization, and other overhead. There may also be a small amount of work that cannot be perfectly parallelized.

Therefore, the results support the prediction that additional workers would reduce elapsed time for this workload, while also showing that real parallel performance is not perfectly ideal.

Limitations

This experiment had several limitations. Only three worker counts were tested, so the results do not show what would happen with worker counts greater than four. The experiment also used only three trials per condition, which limits the amount of information available about timing variation.

Other processes running on the system may have affected scheduling and elapsed time. For example, while running this experiment, Firefox (with multiple tabs) along with Discord were open during the experiment. These programs can use quite a lot of CPU so scheduling could have possibly been affected. In addition, the workload was relatively short, so thread-creation and scheduling overhead may have represented a noticeable portion of the total execution time.

The conclusions apply to the tested program, workload, worker counts, and recorded system conditions. They should not be interpreted as meaning that four workers are *always* faster for every workload or system.